

Print Agent & Label Designer

Install and run the print agent, connect a TSC TE244, and print labels from the Label Designer.

Label Designer (browser)

└─ **list agents:** browser → relay (EC2 :3001)

└─ **send print:** browser → backend → relay → **AGENT** → TE244

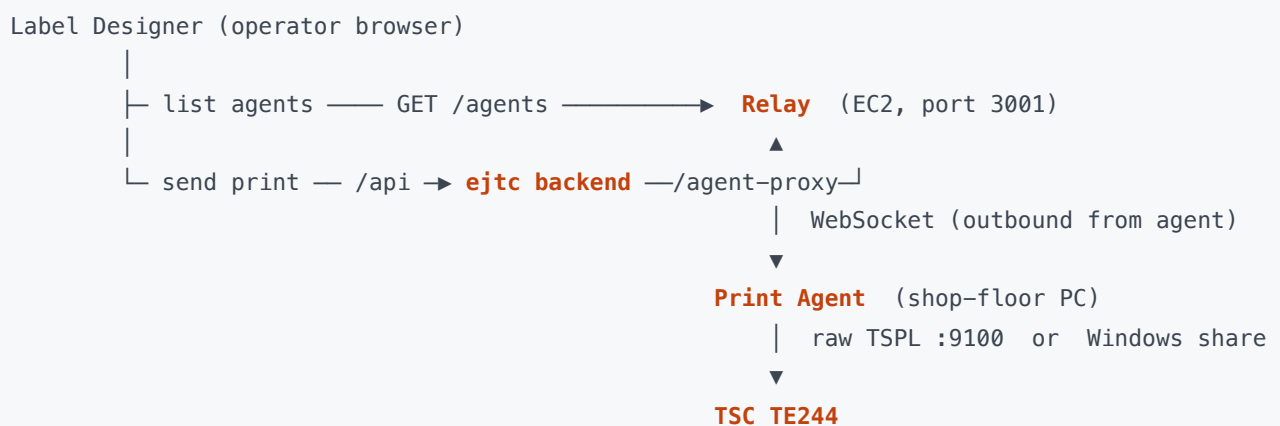
Document	Version	Updated	Printer
Print Agent Handbook	1.1	17 Jul 2026	TSC TE244 · TSPL2 · 203 dpi

Contents

1	How the pieces fit together	the print path
2	Prerequisites	Windows box
3	Install the agent	clone · install
4	Configure config.json	network vs USB
5	Run the agent	start · boot
6	Verify the connection	/agents · /health
7	Print from the Label Designer	step by step
8	Agent list configuration	VITE_RELAY_URL
9	Troubleshooting	symptom → fix
10	Reference	ports · config · files

01 How the pieces fit together

The **agent** is the only component inside the shop LAN that touches the printer. It connects **outbound** to the relay, so no inbound firewall change is needed on the plant network.



KEY IDEA – THE GATEWAY MODEL

You do **not** need an agent on every terminal. Run **one** always-on agent box with the printer; every console routes its print jobs to that agent through the relay. Per-terminal agents are only needed where a terminal has its own attached printer or serial scale.

02 Prerequisites

On the Windows PC that has (or can reach) the printer:

- **Node.js LTS** and **git** installed.
- Ability to build native modules – `npm install` compiles `serialport`, `canvas`, `sharp`, and downloads Chromium for `puppeteer`. Let it finish; your existing terminals already do this.
- **Windows** – the printer status probe and USB-share printing use PowerShell and `copy`.
- Network reachability to the relay on `EC2 :3001` (outbound only).

03 Install the agent

```
:: Clone into the standard location, install deps, seed the config
git clone https://github.com/lincheewei/print-agent.git C:\print-agent
cd /d C:\print-agent
npm install
copy config.example.json config.json
```

The clone location `C:\print-agent` is assumed by the boot scripts (`start-print-agent.bat`), the scheduled task, and the auto-update runner). `config.json` is git-ignored and is never overwritten by updates.

04 Configure `config.json`

Pick **one** printer block for the TE244. The agent exits on boot if `config.json` is missing.

RECOMMENDED Networked TE244 – no Windows driver

```
{
  "agentId": "warehouse-gateway-01",
  "relayUrl": "ws://ec2-43-217-35-209.ap-southeast-5.compute.amazonaws.com:3001",
  "printer": { "tscIp": "192.168.1.87" },
  "localHttpPort": 9000
}
```

Raw TSPL is sent to `tcp://tscIp:9100`. Requires the TE244 on the LAN with **RAW port 9100 enabled** (default). If both `tscIp` and `tscShareName` are set, `tscIp` wins.

ALTERNATIVE USB-only TE244 – via Windows share

Install the TSC driver, **share** the printer in Windows, then reference the share name:

```
{
  "agentId": "warehouse-gateway-01",
  "relayUrl": "ws://ec2-43-217-35-209.ap-southeast-5.compute.amazonaws.com:3001",
  "printer": { "tscShareName": "TSC_TE244" },
  "localHttpPort": 9000
}
```

CONFIG NOTES

`agentId` must be unique per box. `scale` is optional — omit it entirely if this box has no serial scale; the agent runs fine without one. `localHttpPort` defaults to `9000`.

05 Run the agent

MODE	COMMAND	USE FOR
Foreground test	<code>node server.js</code>	First run / debugging (there is no <code>npm start</code>)
Boot script	<code>start-print-agent.bat</code>	Launch + append to <code>agent.log</code>
Service (PM2)	<code>pm2 start ecosystem.config.js</code>	Always-on with auto-restart
Fleet auto-update	push to <code>main</code>	Self-hosted runners — see RUNNER-SETUP.md

06 Verify the connection

```
:: On the agent box — local HTTP server is up
curl http://localhost:9000/health

:: From any machine that reaches EC2 — the agent shows in the relay list
curl http://ec2-43-217-35-209.ap-southeast-5.compute.amazonaws.com:3001/agents
```

`/agents` should return `{"agents":["warehouse-gateway-01"], ...}`. The agent log shows `Connecting to ws://...` then `Agent warehouse-gateway-01 running on http://localhost:9000`.

07 Print from the Label Designer

- 1 Open the console and **unlock** with the supervisor PIN.
- 2 Go to the **Label Designer** tab. Choose a **process** (top-right dropdown), then **New from current layout** or open a saved template. Edit as needed.
- 3 **Save** — required. Test Print always uses the saved version.

- 4 Click  **Test print**, then **pick your agent** from the list.
- 5 **Preview TSPL (dry run)** to sanity-check the commands, then **Send test print** — the label prints on the TE244 through that agent.

GOOD TO KNOW

Activate is only for making a design the *live operator* default (work-order-assign / work-order-scan) — it is **not** required for a test print. The  **Proof** button (PNG / PDF) needs no printer or agent at all — use it to verify layout anywhere.

08 Agent list configuration

The designer lists agents by calling the relay **directly from the browser** at `VITE_RELAY_URL`, baked in at **build time**. Test Print shows **only currently-connected agents** — pick one to send.

✓ ALREADY CONFIGURED ON THIS DEPLOYMENT

`VITE_RELAY_URL` is set in the console's `.env` on the box to `http://ec2-43-217-35-209...:3001` — the same relay the operator kiosk already uses to print, so **:3001 is reachable from the shop floor and no security-group change was needed**. Rebuilds keep it (the value persists in `.env`).

If the agent list is ever **empty** when an agent is up, check, in order:

- the console was built *with* `VITE_RELAY_URL` (it's in `warehouse-console/.env`, git-ignored — a fresh clone needs it re-created);
- the browser can reach `VITE_RELAY_URL/agents` (EC2 SG inbound **:3001** open to that network — it is `http/ws`, keep it to the LAN/VPN, never `0.0.0.0/0`);
- the agent is actually registered — `curl <relay>:3001/agents`.

The *send* step goes browser → backend → relay → agent; only the agent *list* depends on the browser reaching the relay directly.

09 Troubleshooting

SYMPTOM	CAUSE / FIX
"No agents connected" in Test Print, but the agent is up	<code>VITE_RELAY_URL</code> not baked into the console build, or SG :3001 blocked — see §08
<code>/agents</code> returns empty	Agent <code>relayUrl</code> wrong, or the agent box can't reach EC2 :3001 outbound
Send fails with "Agent timeout"	Agent connected but the print didn't return within 6s — printer unreachable / offline
Network print fails	TE244 not on the LAN, RAW port 9100 disabled, or wrong <code>tscIp</code>
USB print fails	Driver not installed, printer not shared, or wrong <code>tscShareName</code>
<code>npm install</code> errors	Missing native build tools for serialport / canvas / sharp / puppeteer
Agent exits immediately	<code>config.json</code> missing — copy it from <code>config.example.json</code>

10 Reference

Ports

PORT	WHERE	PURPOSE
3001	EC2 (relay)	Agent WebSocket + <code>/agents</code> , <code>/agent-proxy</code>
9000	Agent box	Agent local HTTP (<code>/health</code> , <code>/print-label</code>)
9100	Printer	Raw TSPL (RAW / JetDirect) for networked TE244

config.json fields

FIELD	REQUIRED	NOTES
<code>agentId</code>	Yes	Unique identity shown in the agent list
<code>relayUrl</code>	Yes*	Optional to boot, but required to receive print jobs
<code>printer.tscIp</code>	One of	Network path — raw TSPL to :9100 (wins over share)
<code>printer.tscShareName</code>	One of	USB path — Windows shared printer
<code>localHttpPort</code>	No	Defaults to 9000
<code>scale.port</code>	No	Serial scale COM port; omit if none

Key files

FILE	ROLE
<code>print-agent/server.js</code>	The agent (relay client + print dispatch)
<code>print-agent/config.json</code>	Per-box config (git-ignored)
<code>print-agent/RUNNER-SETUP.md</code>	Auto-update runners + gateway model
<code>ejtc-warehouse/.../relay-server.cjs</code>	Relay (EC2, pm2 <code>relay</code>)

TE244 FIT

The TE244 speaks **TSPL2 at 203 dpi** — exactly what the Label Designer emits. No firmware, driver, or label-code change is required; only the connection (network IP vs USB share) needs deciding.